

Logistic Regression – Complete Notes

Definition, use cases, advantages, disadvantages, mathematical idea, scikit-learn model, arguments, prediction, evaluation, and practical example.

1. What is Logistic Regression?

Logistic Regression is a supervised machine learning algorithm used mainly for **classification** problems. It predicts the probability that an observation belongs to a particular class.

For binary classification, the output is usually represented as 0 or 1. The model first calculates a probability between 0 and 1 and then applies a classification threshold, commonly 0.5.

Examples: spam/not spam, customer will purchase/will not purchase, pass/fail, disease/no disease, loan default/no default.

2. Why is it called Logistic Regression?

Although its name contains the word regression, Logistic Regression is generally used for classification. It uses a linear combination of input features and passes the result through the logistic (sigmoid) function.

The sigmoid function converts the model's score into a value between 0 and 1, which can be interpreted as a probability for the positive class.

Probability: $P(y=1|x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)})$

3. When Do We Use Logistic Regression?

Situation	Example
Binary classification	Purchased = 0/1
Email classification	Spam = 1, Not Spam = 0
Medical classification	Disease = 1, No disease = 0
Banking	Default = 1, No default = 0
Marketing	Customer responds = 1, No response = 0
HR	Employee leaves = 1, Stays = 0

4. Advantages

- Simple and easy to understand.
- Fast to train and fast to make predictions.
- Works well as a baseline classification model.
- Produces probabilities, not only class labels.
- Coefficients can provide useful information about feature effects.
- Works well when the relationship between predictors and log-odds is approximately linear.
- Supports regularization to help control overfitting.

5. Disadvantages

- Assumes a linear relationship between features and the log-odds of the target.
- Can perform poorly when the decision boundary is highly nonlinear.
- Sensitive to strong multicollinearity among features.
- Outliers can affect the estimated coefficients.
- Feature scaling is often useful, especially when regularization is used.
- For complex patterns, tree-based or nonlinear models may perform better.

6. Binary vs Multi-class Logistic Regression

Binary: Two classes, such as 0/1 or Yes/No. Example: customer purchases or does not purchase.

Multi-class: More than two classes. For example, classifying a flower as Setosa, Versicolor, or Virginica. Scikit-learn's LogisticRegression supports multi-class classification.

7. Logistic Regression Model in Python

The commonly used scikit-learn model is:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

8. Important LogisticRegression() Arguments

Argument	Meaning	Example
penalty	Type of regularization.	penalty='l2'
C	Inverse of regularization strength. Smaller C means stronger regularization	C=1.0
solver	Optimization algorithm used to fit the model.	solver='lbfgs'
max_iter	Maximum number of optimization iterations.	max_iter=100
class_weight	Weights classes differently; useful for imbalanced data.	class_weight='balanced'
random_state	Controls randomness where applicable.	random_state=42
fit_intercept	Whether to calculate an intercept.	fit_intercept=True
tol	Tolerance used to decide convergence.	tol=0.0001

9. Detailed Explanation of Important Arguments

penalty — Regularization helps control model complexity. Common choices include l2 and, with compatible solvers, l1. Use None to disable regularization when appropriate.

C — C is the inverse of regularization strength. A smaller C applies stronger regularization; a larger C applies weaker regularization.

solver — The solver is the numerical optimization method. lbfgs is a common default choice for many problems. Other options include liblinear, newton-cg, newton-cholesky, sag, and saga, with different compatibility rules.

max_iter — The maximum number of iterations allowed for optimization. If the model does not converge, increasing max_iter can help.

class_weight — Controls the weight assigned to classes. 'balanced' automatically adjusts weights based on class frequencies and can help with imbalanced targets.

random_state — Sets a reproducible seed where the chosen solver or procedure uses randomness.

fit_intercept — If True, the model learns an intercept (bias) term β_0 .

tol — Convergence tolerance. A smaller tolerance can require more iterations but may produce a more tightly converged solution.

10. Complete Practical Example

Suppose we have a dataset with Age, Salary, Experience and Purchased. Purchased is the target: 0 = No, 1 = Yes.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv("logistic_regression_small.csv")
```

```

X = df[["Age", "Salary", "Experience"]]
y = df["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(
penalty="l2",
C=1.0,
solver="lbfgs",
max_iter=1000,
random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

11. Important Model Attributes and Methods

Item	Purpose
model.fit(X_train, y_train)	Train the model.
model.predict(X_test)	Predict class labels.
model.predict_proba(X_test)	Return class probabilities.
model.coef_	Learned coefficients for features.
model.intercept_	Learned intercept.
model.classes_	Class labels known by the model.

12. Prediction Example

If a trained model gives a probability of 0.82 for Purchased=1, then with the usual 0.5 threshold the predicted class is 1. If the probability is 0.30, the predicted class is 0. The threshold can be changed depending on the application.

13. Evaluation Metrics

- **Accuracy:** Percentage of predictions that are correct.
- **Precision:** Among predicted positives, how many are actually positive.
- **Recall:** Among actual positives, how many were detected.
- **F1-score:** Harmonic mean of precision and recall.
- **Confusion Matrix:** Shows TP, TN, FP and FN.
- **ROC-AUC:** Measures ranking/discrimination performance across classification thresholds.

14. Key Interview Points

- Logistic Regression is primarily a classification algorithm.
- The sigmoid function maps a linear score to a probability between 0 and 1.
- A threshold converts probability into a class label.
- Regularization helps reduce overfitting.
- StandardScaler is often used when feature magnitudes differ and regularization is applied.
- `predict()` returns class labels; `predict_proba()` returns probabilities.
- For imbalanced classes, evaluate more than accuracy and consider `class_weight='balanced'` when appropriate.